

Claude Code 企業級 Java + React 自動化開發完整指南

適用技術棧：Spring Boot 3.x + Vavr + PostgreSQL/Citus + 多租戶架構（後端） | React + TypeScript（前端） 核心原則：CLAUDE.md + 自建 Skills 的品質 > 堆疊外掛數量；Superpowers 管方法論，OpenSpec 管規格文件，ralph-wiggum 管自主執行，skill-creator 管技能生命週期。

一、第一性原理：為什麼需要這套架構

問題本質

AI 代理寫程式的三大系統性風險：

- 1. 跳過設計直接寫 code——缺乏規格文件，需求理解偏差在後期才爆發
- 2. 不寫測試就宣告完成——AI 傾向宣稱成功而非驗證成功
- 3. 上下文膨脹導致品質退化——長 session 中 token 耗盡，後半段輸出品質崩潰

解法拆解

風險	對應機制	具體工具
跳過設計	規格驅動開發（SDD）	OpenSpec 產出 spec → Superpowers brainstorming 硬門檻
不寫測試	TDD 強制執行	Superpowers test-driven-development（先寫測試，違反就刪 code）
上下文膨脹	Subagent 隔離 + context 壓縮	Superpowers v5.0 每個任務獨立 subagent + <code>/compact</code>
品質不一致	確定性品質關卡	Hooks（PostToolUse/Stop）+ 自建 Skills
自主執行	迴圈攔截退出	ralph-wiggum Stop hook
技能維護	評估 + 基準測試	skill-creator eval/benchmark

二、OpenSpec 與 Superpowers 對比分析

結論先行：互補而非替代

OpenSpec 無法取代 Superpowers。兩者解決不同層面的問題：

- OpenSpec = 規格文件管理系統（管「做什麼」）
- Superpowers = 開發方法論強制執行框架（管「怎麼做」）

逐維度對比

維度	OpenSpec	Superpowers	判定
需求分析	GIVEN/WHEN/THEN 場景、proposal.md	brainstorming 蘇格拉底式問答	互補：OpenSpec 管文件，Superpowers 管流程
規劃	design.md + tasks.md	writing-plans 微任務（2-5 分鐘粒度）	Superpowers 更細粒度
TDD 強制	❌ 無	✅ 硬門檻：先寫測試才能寫 code，違反直接刪除	Superpowers 獨有
Code Review	spec delta 差異比對	雙階段 subagent 審查（spec 合規 + code 品質）	Superpowers 更強
完成驗證	<code>/opsx:verify</code> 基本 spec 比對	verification-before-completion：必須有測試執行證據	Superpowers 更嚴格
腦力激盪紀律	❌ 無	✅ 硬門檻：設計未核准前禁止寫 code	Superpowers 獨有
系統性除錯	❌ 無	✅ 四階段根因調查（必須調查後才能修復）	Superpowers 獨有
Subagent 編排	❌ 無	✅ subagent-driven-development（v5.0 核心）	Superpowers 獨有
Brownfield 支援	✅ 優秀（spec delta 追蹤既有系統）	一般	OpenSpec 更強
規格文件生命週期	✅ 提案→套用→封存完整流程	基礎（docs/superpowers/specs/）	OpenSpec 更完整
多工具支援	20+ AI 助手	Claude Code + Codex + Cursor + Gemini CLI	OpenSpec 更廣
社群規模	~23,800 星 / 46 貢獻者	~108,000 星 / 234K 安裝	Superpowers 壓倒性領先

OpenSpec 核心機制

OpenSpec 由 Fission-AI 開發，是獨立 CLI 工具（非原生 Claude Code 外掛），透過 `npm install -g @fission-ai/openspec@latest` 安裝。有繁體中文分支 `@minidoracat/openspec-tw`。

核心工作流：**Propose** → **Apply** → **Archive**

```
/opsx:propose → 產出 proposal.md + specs/ + design.md + tasks.md
/opsx:apply → 逐步執行 tasks.md 中的任務
/opsx:verify → 驗證實作是否符合 spec
/opsx:archive → 時間戳封存 + 合併 spec delta
/opsx:ff → 快轉：一次走完所有規劃產物
```

最大優勢是 **spec delta 追蹤**——對既有系統特別有用，能追蹤現有規格與提案規格的差異，讓 code review 聚焦「意圖」而非「diff」。

Superpowers v5.0.6 核心機制

由 Jesse Vincent（Prime Radiant 創辦人）開發，2026 年 1 月進入 Anthropic 官方 Marketplace。最新 v5.0.6（2026-03-25 發布）。

14 個 Skills 形成完整管線：

```
brainstorming → writing-plans → subagent-driven-development
                                     └─ test-driven-development (RED-GREEN-REFACTOR)
                                     └─ systematic-debugging (四階段根因調查)
                                     └─ verification-before-completion (證據先於斷言)
                                     └─ requesting/receiving-code-review
→ finishing-a-development-branch
```

關鍵強制機制：

- **<HARD-GATE>** 標記：字面上阻止實作直到前置條件滿足
- 合規測試：使用 Cialdini 說服原則（權威、承諾、稀缺、社會認同）進行對抗性壓力測試
- v5.0 subagent 模式：每個任務啟動獨立 implementer subagent → spec-compliance reviewer → code-quality reviewer
- v5.0.6 優化：移除 brainstorming/planning 的 subagent 審查迴圈（每 session 省 ~25 分鐘）

Token 成本：核心引導 ~2,000 tokens；全部 Skills 載入 ~22,000 tokens（200k context 的 ~11%）。

三、最終外掛安裝清單（移除 Deep Trilogy）

第一層：規格管理

工具	安裝方式	用途
OpenSpec	<code>npm install -g @fission-ai/openspec@latest</code>	規格驅動開發：Propose → Apply → Archive

工具	安裝方式	用途
OpenSpec 繁體中文	<code>npm install -g @minidoracat/openspec-tw</code>	繁體中文 prompt (可選)

第二層：方法論強制

外掛	安裝指令	用途
superpowers	<code>/plugin install superpowers</code>	14 Skills：brainstorming 硬門檻、TDD 強制、雙階段 review、subagent 編排

第三層：自主執行引擎

外掛	安裝指令	用途
ralph-wiggum	<code>/plugin install ralph-wiggum</code>	Stop hook 攔截退出，自主迭代迴圈

第四層：技能生命週期管理

外掛	安裝指令	用途
skill-creator	<code>/plugin install skill-creator</code>	建立、評估、基準測試、版本管理自建 Skills

第五層：Java 後端

外掛	安裝指令	用途
developer-kit-java	<code>/plugin marketplace add giuseppe-trisciuoglio/developer-kit</code> → <code>/plugin install developer-kit-java</code>	11 Commands + 9 Agents + 52 Skills (CRUD、JWT、Saga、Security)
context7	<code>/plugin install context7</code>	Spring Boot 3.x / Vavr 版本精準文件查詢
java-lsp	<code>/plugin install java-lsp</code>	Eclipse JDT LS：Go to Definition、Diagnostics

第六層：React 前端

外掛	安裝指令	用途
frontend-design	<code>/plugin install frontend-design</code>	產品級 UI 設計指導 (371K 安裝)
typescript-lsp	<code>/plugin install typescript-lsp</code>	TypeScript 型別檢查

第七層：安全與品質

外掛	安裝指令	用途
security-guidance	<code>/plugin install security-guidance</code>	命令注入、XSS、不安全模式監控
code-review	<code>/plugin install code-review</code>	4 Agent 平行 PR 審查

第八層：MCP 伺服器

```
bash

# PostgreSQL/Citus (唯讀連線)
claude mcp add postgres -- npx @modelcontextprotocol/server-postgres \
  postgresql://readonly:password@localhost:5432/gaming_platform

# Context7 文件查詢
claude mcp add --scope user context7 -- npx -y @upstash/context7-mcp
```

四、自建 Skills + skill-creator 維護體系

自建 Skills (填補外掛生態空白)

三個核心 Skills 對應你的技術棧中沒有現成外掛覆蓋的領域：

```
.claude/skills/vavr-patterns/SKILL.md
```

```
yaml
```

```

---
name: vavr-patterns
description: >
  Vavr 函數式程式設計模式。處理錯誤處理、Either/Try 鏈式操作、
  不可變集合、模式比對時自動使用。適用於 Service 層方法返回值
  重構和 Controller 錯誤處理轉換。
allowed-tools: "Read,Write,Edit,Bash,Grep,Glob"
version: "1.0.0"
---

# Vavr 函數式模式規範

## 核心規則
- Service 層返回 `Either<AppError, T>`，禁止 checked exception
- 使用 `Try.of()` 替代 try-catch 區塊
- Domain 集合使用 `io.vavr.collection.List` (非 java.util.List)
- Controller 回傳 Either 時搭配 `fold()` 轉換 ResponseEntity
- 必須配置 VavrModule 於 Jackson ObjectMapper (vavr-jackson)
- Pattern Matching 使用 `Match(value).of(Case($(...), ...))` 語法
- Optional 一律改用 Vavr Option
- 禁止直接返回 null

## 錯誤處理模板
```java
public Either<AppError, OrderResponse> createOrder(CreateOrderRequest req) {
 return validateRequest(req)
 .flatMap(this::checkBalance)
 .flatMap(this::deductBalance)
 .flatMap(this::persistOrder)
 .peek(order -> eventPublisher.publish(new OrderCreatedEvent(order)));
}
```

## 驗證腳本
完成後執行：`./mvnw test -pl :目標模組` 確認所有測試通過

```

.claude/skills/citus-multitenant/SKILL.md

yaml

```
---
name: citus-multitenant
description: >
  Citus 分散式 PostgreSQL 多租戶架構模式。處理資料庫 Schema、
  Liquibase 遷移、Repository 查詢、JOIN 操作時自動使用。
  確保所有查詢包含 tenant_id 過濾。
allowed-tools: "Read,Write,Edit,Bash,Grep,Glob"
version: "1.0.0"
---

# Citus 多租戶架構規範

## 分片規則
- 所有租戶表必須包含 tenant_id 作為分片鍵 (distribution column)
- `SELECT create_distributed_table('table_name', 'tenant_id')`
- Reference tables 用 `create_reference_table()` (game_types、currency_codes)

## 查詢規則
- 所有 WHERE 子句必須包含 tenant_id (確保 co-location)
- JOIN 兩表都必須含 tenant_id 在 JOIN 條件中
- 禁止跨租戶查詢 (除非透過 coordinator node)

## Liquibase 規則
- 先 CREATE TABLE 再 distribute (Citus DDL 限制)
- 禁止 autoIncrement (跨節點不支援，改用 UUID/Snowflake ID)
- 新增索引必須 CREATE INDEX CONCURRENTLY
- changeSet id 格式：YYYYMMDD-序號-描述

## 併發規則
- 零悲觀鎖：@Version 樂觀鎖 + tenant_id 複合唯一約束
- Row-Level Security 作為額外隔離層
```

.claude/skills/gaming-domain/SKILL.md

yaml

```
---
name: gaming-domain
description: >
  iGaming 平台領域知識。處理錢包（入金/出金/投注扣款/派彩）、
  VIP 等級、風控規則、投注單據相關業務邏輯時自動使用。
allowed-tools: "Read,Write,Edit,Bash,Grep,Glob"
version: "1.0.0"
---

# iGaming 領域規範

## 金融操作
- 錢包操作必須保證 exactly-once 語義（冪等鍵 + 狀態機）
- 投注金額使用 BigDecimal，禁止 double/float
- 所有金融操作必須有完整 audit trail (Kafka event sourcing)
- API 回應必須包含 requestId 用於追蹤

## 效能要求
- P95 延遲 < 100ms
- 批次投注使用 saveAll() 非迴圈 save()
- 使用 @Cacheable 於讀取密集操作

## 事件驅動
- VIP 等級變更透過 Kafka 事件驅動更新，非批次處理
- 風控規則透過 Evrete 規則引擎執行，非硬編碼
```

skill-creator 維護工作流

Anthropic 官方外掛，提供技能全生命週期管理：


```
# 建立新技能（互動式引導）
> 使用 skill-creator 建立一個 PITest 突變測試品質關卡技能

# 評估技能效果（平行 subagent 對照）
> /skill-creator eval vavr-patterns
  → 啟動兩個 subagent：一個載入技能、一個不載入
  → 產出 HTML 比較報告：baseline vs. enhanced

# 改善技能（基於 eval 失敗案例）
> /skill-creator improve vavr-patterns
  → 分析 eval 失敗案例
  → 60/40 train/test split 優化 description
  → 防止過擬合

# 基準測試（統計分析）
> /skill-creator benchmark vavr-patterns --runs 5
  → 多次執行的變異數分析
  → 版本釘選的結果追蹤
```

核心價值：Skills 需要跟生產 code 一樣的嚴格度——版本控制、測試驗證、受控部署。skill-creator 提供了這個閉環。

五、Hooks 品質關卡配置

```
json
```

```
{
  "permissions": {
    "allow": [
      "Read", "Write", "Edit", "Glob", "Grep",
      "Bash(./mvnw test*)", "Bash(./mvnw spotless:apply*)",
      "Bash(./mvnw checkstyle:check*)", "Bash(./mvnw spotbugs:check*)",
      "Bash(npm run test*)", "Bash(npm run build*)", "Bash(npx tsc*)",
      "Bash(git add*)", "Bash(git commit*)", "Bash(git diff*)", "Bash(git status*)",
      "Bash(**/ralph-wiggum/**)"
    ],
    "deny": [
      "Bash(rm -rf*)", "Bash(./mvnw deploy*)", "Bash(docker push*)",
      "Bash(kubectrl*)", "Bash(git push*)", "Bash(DROP *)", "Bash(TRUNCATE*)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "jq -r '.tool_input.file_path // empty' | grep '\\.java$' | xargs"
        }]
      },
      {
        "matcher": "Edit|Write",
        "hooks": [{
          "type": "command",
          "command": "jq -r '.tool_input.file_path // empty' | grep -E '\\.(tsx?|jsx"
        }]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": "echo $CLAUDE_TOOL_INPUT | grep -qiE '(DROP TABLE|TRUNCATE|DELE"
        }]
      }
    ],
    "Stop": [
      {
        "hooks": [{
          "type": "command",
          "command": "echo '--- 後端測試 ---' && ./mvnw test -q 2>&1 | tail -5; echo"
        }]
      }
    ]
  }
}
```

```
    }],
  },
  {
    "hooks": [{
      "type": "command",
      "command": "curl -s -X POST 'https://api.telegram.org/bot$TG_BOT_TOKEN/send"
    }]
  }
]
```

Hook 類型說明

| 類型 | 觸發方式 | 用途 | Token 成本 |
|---------|---------------------------------|--------------------------|----------|
| command | shell 命令（exit 0 通過 / exit 2 阻擋） | 格式化、lint、危險操作攔截 | 零 |
| prompt | 單輪 AI 評估（預設 Haiku） | 快速合規檢查 | 低 |
| agent | 多輪 subagent（可用工具） | 深度審查（N+1 查詢、tenantId 遺漏） | 高 |
| http | POST 到 URL 端點 | 外部系統整合 | 零 |

六、Ralph-wiggum + Superpowers + OpenSpec 聯合工作流

從需求到交付的完整管線

Phase 0: 環境準備

- └─ 安裝外掛（依上方清單）
- └─ 建立自建 Skills (Vavr/Citus/Gaming)
- └─ 配置 Hooks (settings.json)
- └─ 配置 CLAUDE.md（專案規則）

Phase 1: 規格定義 (OpenSpec 主導，人工 30-60 分鐘)

- └─ /opsx:propose → 產出 spec + design + tasks
- └─ 人工審查 spec 文件
- └─ 確認 Vavr Either/tenantId/樂觀鎖要求正確描述

Phase 2: 設計精煉 (Superpowers brainstorming)

- └─ Superpowers brainstorming skill 啟動
- └─ 蘇格拉底式問答精煉設計
- └─ 硬門檻：設計未核准前禁止寫 code
- └─ writing-plans 產出微任務清單（2-5 分鐘粒度）

Phase 3: TDD 實作 (Ralph-wiggum 自主迴圈)

- |— Ralph-wiggum 啟動自主迭代
- |— 每個任務: Superpowers subagent-driven-development
 - | |— 啟動獨立 implementer subagent
 - | |— TDD: 先寫失敗測試 → 最小實作 → 重構
 - | |— spec-compliance reviewer subagent
 - | |— code-quality reviewer subagent
- |— Stop hook 自動跑測試
- |— 完成或達到迭代上限

Phase 4: 品質驗證 (自動 + 手動)

- |— 自動: Hooks (格式化/lint/測試)
- |— 手動: /devkit.java.code-review
- |— 手動: /devkit.java.security-review
- |— 手動: /devkit.java.architect-review

Phase 5: 封存 + 報告

- |— /opsx:archive → 封存本次變更的 spec
- |— 產出最終報告
- |— 建立 Pull Request

Ralph-wiggum 實戰指令

基本用法

```
/ralph-loop "按照 openspec/specs/ 中的規格逐步實作。  
每個任務遵循 TDD: 先寫失敗測試 → 最小實作 → 重構。  
確保所有 Service 方法返回 Either<AppError, T>。  
確保所有查詢包含 tenantId 過濾。  
每完成一個任務執行 ./mvnw test -pl :wallet-service。  
如果測試通過: git commit。  
如果測試失敗: 分析失敗原因, 修復, 重試。  
全部完成後輸出 <promise>WALLET_DONE</promise>。"  
--max-iterations 30 --completion-promise "WALLET_DONE"
```

取消迴圈

```
/cancel-ralph
```

成本預估

```
# 30 iterations × Sonnet = ~$15-30  
# 30 iterations × Opus = ~$50-100
```

關鍵參數說明

| 參數 | 說明 | 建議值 |
|-----------------------------------|--------------|------------------------------|
| <code>--max-iterations</code> | 硬上限，防止無限迴圈 | 初始 15-20，觀察後調整 |
| <code>--completion-promise</code> | 完成信號（精確字串匹配） | 短且唯一的字串如 <code>"DONE"</code> |

機制原理

Ralph-wiggum 透過 **Stop hook 返回 exit code 2** 阻止 Claude 退出，重新注入原始 prompt。Claude 看到自己之前的修改，繼續迭代。一次「迭代」＝一個完整 Claude session 週期。

社群共識：bash 原生方式（每次迭代獨立 context window）比外掛的單 session 模式更穩健，因為避免了 context 膨脹。Anthropic 已將此模式吸收為原生 `/loop` 指令。

七、Subagents 自訂審查代理

`.claude/agents/spring-reviewer.md`

markdown

```
---
name: spring-reviewer
description: >
    審查 Spring Boot 代碼的架構合規性、安全性、DDD 模式。
    修改 service 或 controller 類別後自動呼叫。
tools: Read, Grep, Glob
model: sonnet
maxTurns: 15
skills:
    - vavr-patterns
    - citus-multitenant
---
```

你是資深 Spring Boot 架構師，審查以下維度：

1. ****Vavr 模式****：是否正確使用 Either/Try，是否有 null 返回
2. ****多租戶隔離****：所有查詢是否包含 tenantId 過濾
3. ****DDD 邊界****：Aggregate Root 是否正確，Repository 模式是否合規
4. ****併發安全****：是否有悲觀鎖 (grep SELECT.*FOR UPDATE)
5. ****N+1 查詢****：是否使用 JOIN FETCH 或 @EntityGraph
6. ****安全合規****：@PreAuthorize、敏感欄位 @JsonIgnore

輸出格式：

-  CRITICAL (阻擋合併)
-  WARNING (應修復)
-  INFO (建議)

.claude/agents/react-reviewer.md

markdown

```
---
name: react-reviewer
description: >
    審查 React TypeScript 元件的型別安全、狀態管理、錯誤處理。
tools: Read, Grep, Glob
model: sonnet
maxTurns: 10
---
```

審查維度：

1. TypeScript any 型別使用 (應消除)
2. 元件是否處理 loading/error/empty 狀態
3. 是否有 prop drilling (應用 Context 或 Zustand)
4. 自訂 Hook 是否正確封裝業務邏輯
5. 是否有未處理的 Promise rejection

八、其他值得關注的框架

| 框架 | 定位 | 與本方案的關係 |
|-------------------|--|---|
| cc-sdd
(rhuss) | 橋接 Spec Kit + Superpowers，
trait 系統 | 提供 <code>superpowers</code> trait 自動在規格/實作間加品質
關卡 |
| BMAD
Method | 模擬完整敏捷團隊（9 AI agent） | 銀行 COBOL→Java Spring Boot 50K LOC 遷移驗
證，適合大型 legacy 專案 |
| GSD | 反儀式：最多 3 任務/plan、每任務
獨立 subagent | 執行效率最高，但治理不足，不適合企業場景 |
| Metaswarm | 9 階段 + BEADS 狀態追蹤 | 基於 Superpowers 擴展，100% test coverage，適
合高保證場景 |

九、Everything Claude Code（ECC）深度對比分析

ECC 概覽

Everything Claude Code（ECC）由 Anthropic 黑客松冠軍 Affaan Mustafa 開發，是目前 Claude Code 生態中規模最大的單一外掛合集，GitHub ~103K 星、48K+ 安裝、769 commits。核心定位為「Agent 調度效能優化系統」——不只是 Skills 集合，而是一套完整的四層架構（互動層 → 智能層 → 自動化層 → 學習層），涵蓋 28 個 Subagent、40+ Skills、32 個快捷指令、以及跨 session 持續學習機制。

安裝方式：

```
bash

/plugin marketplace add affaan-m/everything-claude-code
/plugin install everything-claude-code@everything-claude-code

# 安裝語言專用規則（需 clone repo）
git clone https://github.com/affaan-m/everything-claude-code.git
cd everything-claude-code
./install.sh typescript # 或 python / go lang
```

ECC 四層架構拆解

| 層級 | 組成 | 說明 |
|-----|---------------|--|
| 互動層 | 32 指令 + Rules | <code>/plan</code> 、 <code>/tdd</code> 、 <code>/e2e</code> 、 <code>/code-review</code> 、 <code>/security-scan</code> 、 <code>/multi-plan</code> 、 <code>/evolve</code> 等入口 |

| 層級 | 組成 | 說明 |
|------|----------------------------------|---|
| 智能層 | 28 Agents + 40+ Skills | 階層式委派：Orchestrator (Planner/Architect) → Specialist (TDD/Reviewer/Security) |
| 自動化層 | Hooks + Scripts | PreToolUse/PostToolUse 品質關卡、SessionStart/Stop 上下文管理 |
| 學習層 | Instinct v2 + <div>/evolve</div> | 跨 session 經驗積累：Stop hook 提取模式 → Instinct (信心分 0.3-0.9) → <div>/evolve</div> 聚合為 Skill |

28 個 Agent 的階層委派模型

Orchestrator 級別（廣域工具存取，可分派子 Agent）：

| Agent | 職責 |
|-----------|----------|
| planner | 功能實作任務分解 |
| architect | 系統設計決策 |

Specialist 級別（限定工具存取）：

| Agent | 職責 | 工具權限 |
|-----------------------------------|-------------------|---------------|
| tdd-guide | TDD 流程引導 | Code + Test |
| code-reviewer | 通用品質審查 | 唯讀 |
| security-reviewer | OWASP 安全審計 | 唯讀 |
| build-error-resolver | 建構錯誤修復 | Build + Debug |
| e2e-runner | Playwright E2E 測試 | 測試執行 |
| refactor-cleaner | 死碼清理 | 編輯 |
| doc-updater | 文件維護 | 文件編輯 |
| go-reviewer / go-build-resolver | Go 專用 | 唯讀 / Build |
| python-reviewer | Python PEP 8 | 唯讀 |
| database-reviewer | SQL/ORM/Migration | 唯讀 |
| cpp-reviewer / cpp-build-resolver | C++ 專用 | 唯讀 / Build |

ECC 獨有能力：持續學習系統

ECC 最獨特的功能是跨 session 經驗累積——其他所有外掛（Superpowers、OpenSpec、ralph-wiggum）都沒有這個能力：

- **v1 學習**：Session 結束時 Stop hook 提取編碼模式，存入 `~/.claude/skills/learned/`，覆蓋率 ~50-80%
- **v2 Instinct 系統**：PreToolUse/PostToolUse hooks 100% 覆蓋，每個學習單元帶信心分 0.3-0.9
- `/evolve` 指令：3 個以上相關 Instinct 自動聚合為可重用 Skill

ECC 獨有能力：AgentShield 安全掃描

內建安全掃描器，912 條測試規則、102 條靜態分析規則、98% code coverage，針對 Claude Code 生態特有風險（API key 洩漏、Hook injection、MCP 漏洞）：

```
bash
npx ecc-agentshield scan # 基本掃描
npx ecc-agentshield scan --opus --stream # 深度掃描（使用 Opus）
```

ECC vs Superpowers vs OpenSpec：三方對比

| 維度 | ECC | Superpowers | OpenSpec |
|------------------|--------------------------|------------------------------|----------------------------|
| 定位 | 全能型 Agent 調度平台 | 方法論強制執行框架 | 規格文件管理系統 |
| GitHub 星數 | ~103K | ~108K | ~24K |
| Agent 數量 | 28 個（含多語言專用） | 14 個 Skills（非獨立 Agent） | 無 |
| TDD 強制 | ✅ tdd-guide Agent | ✅ 硬門檻（違反刪 code） | ❌ |
| TDD 強度 | 引導式（Guide） | 強制式（(<HARD-GATE>） | — |
| Brainstorming 紀律 | ❌ 無硬門檻 | ✅ 設計未核准禁止寫 code | ❌ |
| Code Review | code-reviewer Agent（單階段） | 雙階段 subagent（spec + quality） | spec delta 比對 |
| Security | AgentShield（912 規則） | 無內建 | 無 |
| 規格管理 | ❌ 無 | 基礎 spec 目錄 | ✅
Propose→Apply→Archive |

| 維度 | ECC | Superpowers | OpenSpec |
|---------------------|------------------------------------|------------------------------------|----------------------|
| 持續學習 | ✅ Instinct v2 + /evolve | ❌ | ❌ |
| 系統性除錯 | build-error-resolver Agent | systematic-debugging 四階段 | ❌ |
| Subagent 編排 | 階層委派
(Orchestrator→Specialist) | subagent-driven-development (v5.0) | ❌ |
| 多語言支援 | TS/Python/Go/Java/C++ (規則 + Agent) | 語言無關 | 語言無關 |
| Java/Spring Boot 深度 | 基礎 Skill (無 Vavr/Citus) | 語言無關 | 語言無關 |
| Context 效率 | Rules 常駐 ~5-8K + Skills 按需 | 核心 ~2K + 全載入 ~22K | 極低 |
| ralph-wiggum 整合 | 無正式整合 | 無正式整合 (互補) | ralphy-openspec 社群橋接 |
| Token 成本 | 高 (28 Agent 分派開銷) | 中 (subagent 隔離省 context) | 低 |

對你的 Spring Boot + Vavr + React 技術棧的適用性分析

ECC 的優勢：

1. 即開即用的廣度——28 Agent 覆蓋 TDD、code review、security scan、e2e test、build fix、文件更新，不需逐個外掛安裝
2. 持續學習系統——Vavr Either 模式、Citus tenant_id 規則等專案特有慣例可自動積累為 Instinct，後續 session 自動套用
3. AgentShield——iGaming 平台的安全要求高，內建 912 規則掃描器是顯著加值
4. 多語言 Agent——如果團隊有 Go/Python 微服務，不需額外配置

ECC 的不足（對你的場景）：

1. 無方法論硬門檻——ECC 的 TDD 是「引導」而非「強制」，不像 Superpowers 會刪掉先寫的 code；brainstorming 無設計核准門檻
2. 無規格文件管理——沒有 OpenSpec 的 Propose→Apply→Archive 生命週期
3. Java/Spring Boot 深度不足——沒有 developer-kit-java 的 11 Commands (`/devkit.java.architect-review` 等)、沒有 Vavr/Citus 專用 Skill
4. Context 消耗大——28 Agent + 常駐 Rules ~5-8K，多 Agent 分派產生額外 API 呼叫成本

5. **Superpowers 的雙階段 review 更嚴格**——ECC 只有 code-reviewer 單階段，Superpowers 拆成 spec-compliance + code-quality 兩個獨立 subagent

結論：ECC 是「瑞士刀」，Superpowers 是「手術刀」

| 選擇 | 適用場景 |
|---|--|
| ECC 單獨使用 | 個人開發者、快速原型、多語言專案、安全敏感專案 |
| Superpowers 單獨使用 | 需要嚴格方法論紀律的團隊、企業級品質要求 |
| ECC + Superpowers 並用 | ⚠️ 不建議——context 消耗過大（~27-30K tokens 常駐），Agent 職責重疊（TDD/review） |
| 本方案（Superpowers + OpenSpec + 自建 Skills） | ✅ 推薦——企業級 Java 技術棧、需要方法論強制 + 規格管理 + 領域專用 Skills |

對你的專案的最終建議：

- 不採用 ECC 作為主框架——方法論強度不足，Java 深度不夠，與 Superpowers 存在大量職責重疊
- 可選擇性借鑑的 ECC 能力：
 - 持續學習系統的 Instinct 概念 → 用 skill-creator 自建類似的 Vavr/Citus 學習 Skill
 - AgentShield 安全掃描 → 可單獨安裝 `npx ecc-agentshield scan` 作為 CI 環節，不需要整個 ECC
 - 階層委派的 Agent 設計模式 → 參考其 planner→specialist 委派結構設計你的自建 Subagent

十、缺漏分析與自建補充

| 缺口 | 現狀 | 補充方式 |
|------------------------|-------|---|
| Vavr 專用外掛 | 不存在 | 自建 Skill + skill-creator eval 驗證 |
| Citus 專用外掛/MCP | 不存在 | 自建 Skill + 標準 PostgreSQL MCP |
| Kafka 測試 | 部分覆蓋 | Testcontainers Kafka 配置加入 devkit 整合測試 |
| Liquibase Migration 審查 | 無專門外掛 | <code>.claude/rules/liquibase.md</code> |
| ArchUnit 架構測試 | 無專門外掛 | CLAUDE.md 規則 + CI 管線 |
| 效能測試 (Gatling) | 無專門外掛 | 自建 Skill 定義效能基準 |

| 缺口 | 現狀 | 補充方式 |
|----------------|-------|----------|
| Vue → React 遷移 | 無專門外掛 | 自建 Skill |

`.claude/rules/liquibase.md`

```
yaml
---
description: Liquibase migration 規則，操作 migration 文件時自動套用
globs: ["**/db/changelog/**", "**/*.sql", "**/migration/**"]
---
- Migration 文件必須可回滾（每個 changeSet 都有 rollback 區塊）
- 禁止生產環境直接 DROP COLUMN（先 deprecated，下個版本再刪）
- 新增索引必須 CREATE INDEX CONCURRENTLY
- 所有表必須有 created_at, updated_at, tenant_id, version 欄位
- Citus 先 CREATE TABLE 再 distribute
- changeSet id: YYYYMMDD-序號-描述
- 禁止 autoIncrement (Citus 跨節點不支援，改用 UUID/Snowflake ID)
```

十一、成本控制策略

| 策略 | 方法 |
|-----------------------|--|
| 模型分層 | 規劃/審查用 Opus（品質）、實作用 Sonnet（速度+省錢） |
| Ralph 迭代上限 | 初始 15-20 次，觀察 2-3 次後決定是否增加 |
| 定期 /compact | 長 session 每 30 分鐘執行一次 |
| Subagent 隔離 | Superpowers v5.0 每個任務獨立 subagent，不膨脹主上下文 |
| Skill 延遲載入 | 描述 ~100 tokens，全文使用時才載入 |
| Context7 按需查詢 | 僅在需要 API 文件時觸發，不全域載入 |
| Superpowers v5.0.6 優化 | brainstorming/planning 移除 subagent review 迴圈，省 ~25 min/session |

十二、日常操作速查

| 場景 | 指令 |
|-------|----------------------------|
| 新功能規格 | <code>/opsx:propose</code> |

| 場景 | 指令 |
|----------------|---|
| 快轉規劃 | <code>/opsx:ff</code> |
| 規格驗證 | <code>/opsx:verify</code> |
| 規格封存 | <code>/opsx:archive</code> |
| Ralph 自主實
作 | <code>/ralph-loop "... --max-iterations 20 --completion-promise
"DONE"</code> |
| 取消 Ralph | <code>/cancel-ralph</code> |
| Java CRUD | <code>/devkit.java.generate-crud</code> |
| Java 審查 | <code>/devkit.java.code-review</code> |
| Java 架構審查 | <code>/devkit.java.architect-review</code> |
| Java 安全審計 | <code>/devkit.java.security-review</code> |
| Java 單元測試 | <code>/devkit.java.write-unit-tests</code> |
| Java 整合測試 | <code>/devkit.java.write-integration-tests</code> |
| PR 審查 | <code>/code-review</code> |
| 建立新 Skill | <code>/skill-creator create</code> |
| 評估 Skill | <code>/skill-creator eval <skill-name></code> |
| 壓縮上下文 | <code>/compact</code> |
| 查看費用 | <code>/cost</code> |
| 回退 | <code>ESC ESC</code> 或 <code>/rewind</code> |

核心結論：OpenSpec 管規格文件的生命週期（Propose → Apply → Archive），Superpowers 管開發方法論的強制執行（brainstorming → TDD → review），ralph-wiggum 管自主執行的持續迴圈，skill-creator 管自建 Skills 的品質驗證。四者疊加而非替代，缺一不可。Everything Claude Code（ECC）雖是生態中最大的單一外掛（103K 星），但其「瑞士刀」定位不如 Superpowers 的「手術刀」適合企業級方法論強制——可借鑑其持續學習與 AgentShield 安全掃描概念，但不建議替代核心架構。CLAUDE.md + 自建 Skills 的品質仍然是決定輸出品質的最大單一因素。